UNIVERSIDADE FEDERAL DE SANTA CATARINA - UFSC

Programa de Pós-Graduação em Ciência da Computação (PPGCC)

Disciplina: Segurança - Web, Identidades e Nuvem (INE410124)

Professora: Carla Merkle Westphall

Relatório Técnico - Atividade 5

Mini Aplicação Segura de Prontuários Médicos com OAuth2/OIDC (Keycloak), Assinatura JWS, Cifragem JWE e Acordo de Chaves ECDH-ES

Aluno: Leandro Ricardo Suchecki Verner

Repositório de código-fonte: https://github.com/leandroverner/INE410124

1. Visão geral da solução

O Portal do Médico (Angular, porta 4200) envia laudos ao Servidor de Prontuários (Spring Boot, porta 8081). O médico se autentica no Keycloak (OIDC, code + PKCE) e recebe um access token. O laudo é assinado com ES256 (JWS), e o JWS é cifrado em um JWE com **alg=ECDH-ES** na curva P-256 e **enc=A128GCM**, formando um Nested JWT (sign-then-encrypt).

Fluxo, na ordem em que ocorre:

#	Passo	Quem
1	Backend gera um par EC P-256 estático (nB, PB) na primeira execução e o persiste em keys/.	Backend
2	Login OIDC (code + PKCE) e obtenção do access token (RS256).	Portal / Keycloak
3	GET /api/chave-publica retorna PB.	Portal / Backend
4	Portal gera o par ES256 do médico (privada não extraível, WebCrypto + IndexedDB) e envia a pública em PUT /api/medico/chave-publica (Bearer). O backend a grava no atributo *chave_publica* do usuário no Keycloak (Admin API).	Portal / Backend / Keycloak
5	Portal assina o laudo com a privada ES256, obtendo o JWS (3 partes).	Portal
6	Portal gera o par ECDH-ES efêmero (nA, PA), calcula K = nA·PB, deriva a CEK com o Concat KDF e cifra o JWS, obtendo o JWE (5 partes) com *epk* = PA no header.	Portal
7	POST /api/prontuarios com Authorization: Bearer e Content-Type: application/jose.	Portal
8	Backend valida o access token, calcula K = nB·PA, deriva a CEK, decifra e obtém o JWS.	Backend
9	Backend busca a pública ES256 do médico no Keycloak e valida a assinatura, o *sub*, *iat/exp* e o *jti* (anti-replay). Responde 201 ou 422.	Backend

2. Screenshots da configuração

O ambiente é executado com Docker Compose, em três contêineres Linux: Keycloak, Servidor de Prontuários e Portal do Médico. O Keycloak sobe em modo *start-dev*, e o realm é importado a partir de *keycloak/Telemedicina-Realm.json*. A máquina de desenvolvimento é um macOS com Docker Desktop, portanto os contêineres rodam sobre o kernel Linux da máquina virtual do Docker.

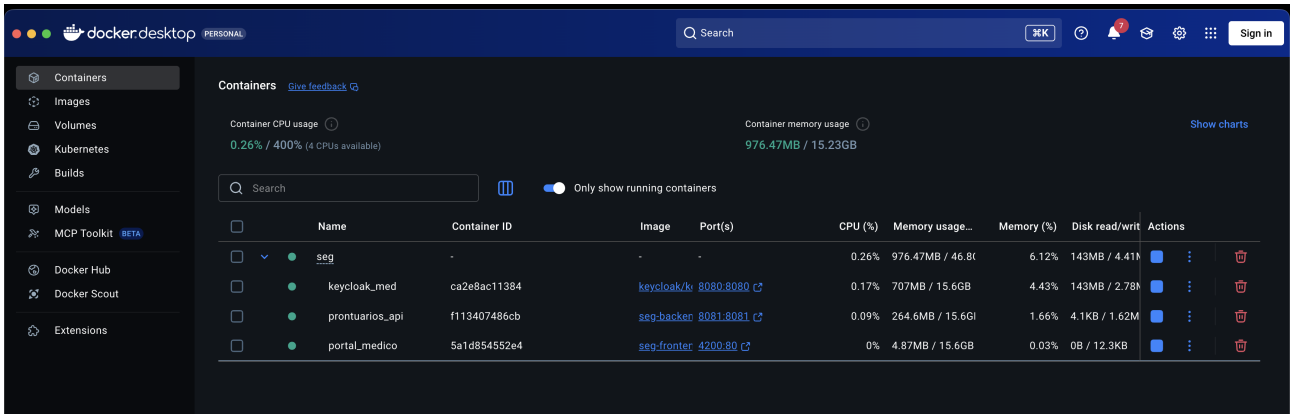


Figura 1 - Docker Desktop com os três contêineres em execução: keycloak_med (porta 8080), prontuarios_api (8081) e portal_medico (4200, mapeada para a porta 80 do contêiner).

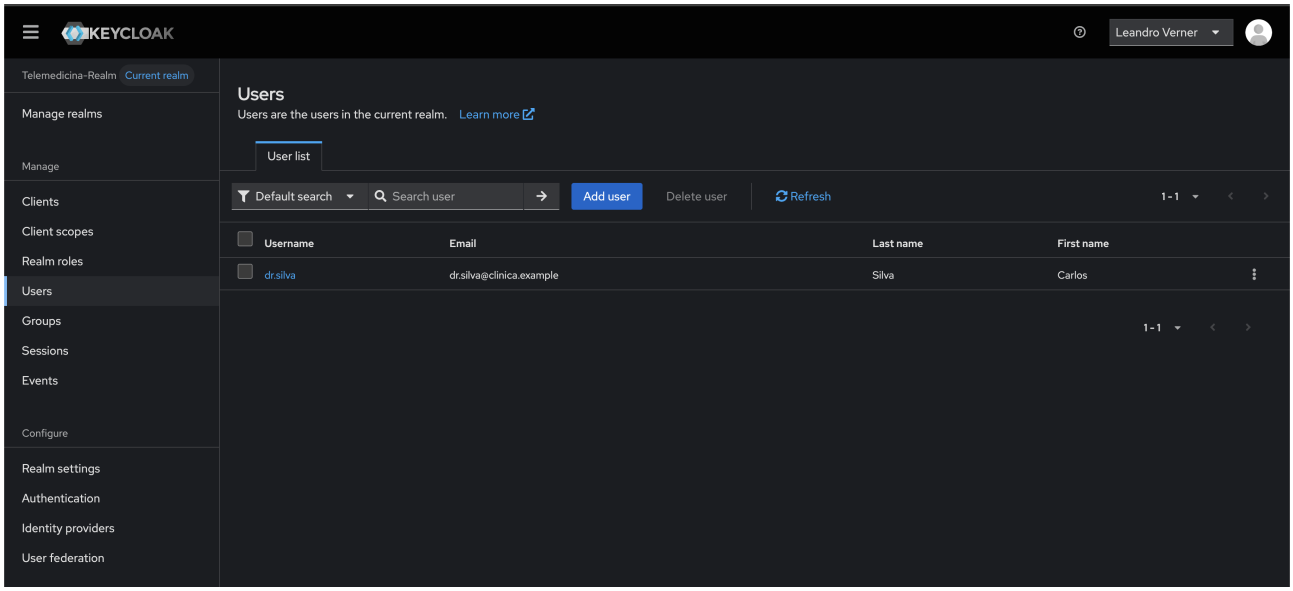


Figura 2 - Realm Telemedicina-Realm no Keycloak, com o usuário de teste dr.silva (Carlos Silva, dr.silva@clinica.example).

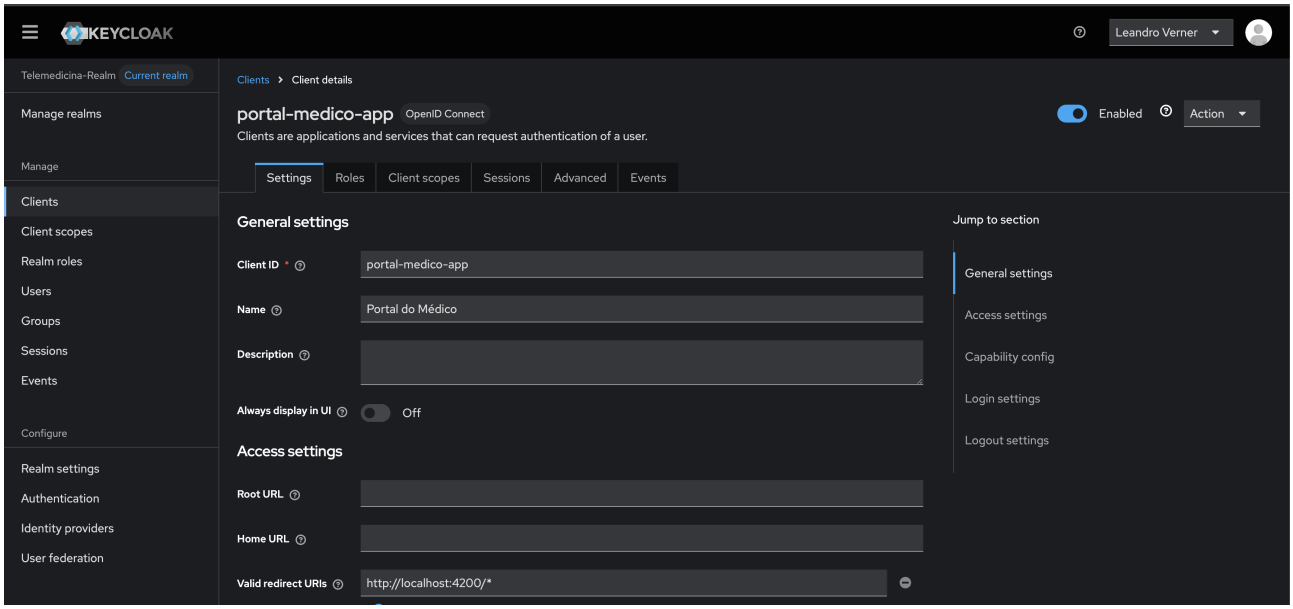


Figura 3 - Client portal-medico-app (OpenID Connect), 'Portal do Médico'. É o client público usado pelo Angular no login OIDC, com Valid redirect URIs = http://localhost:4200/*.

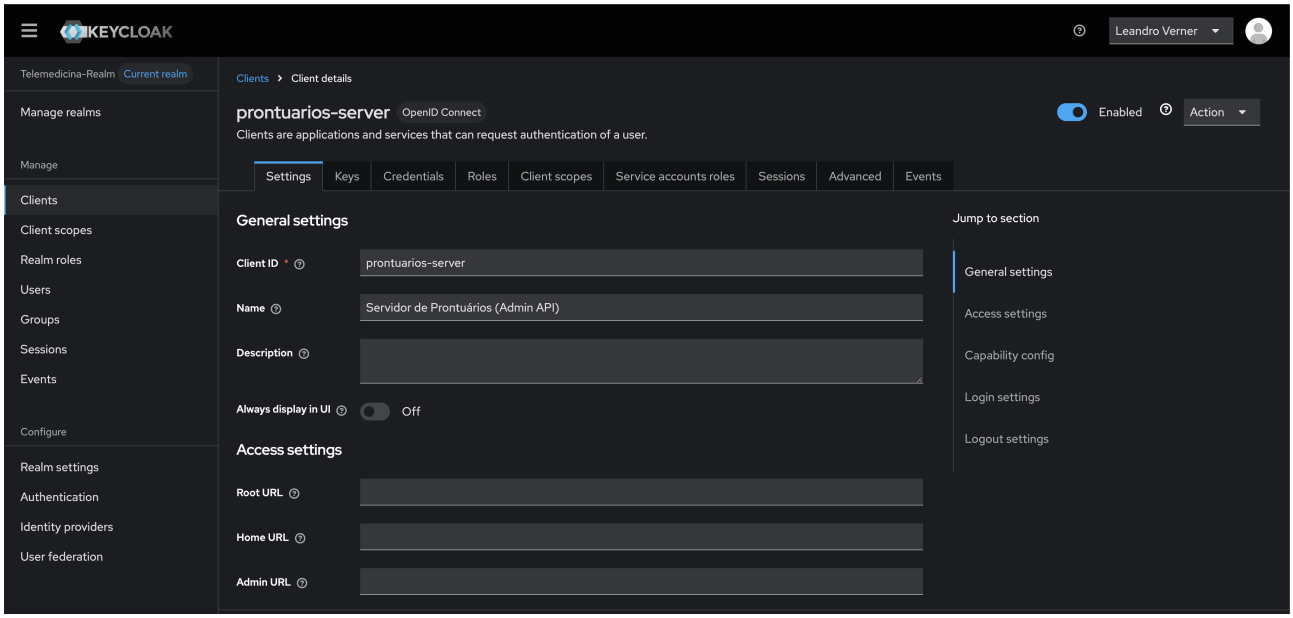


Figura 4 - Client prontuarios-server (OpenID Connect), 'Servidor de Prontuários (Admin API)'. É o client de serviço que o backend usa para gravar e ler o atributo chave_publica do usuário pela Admin API do Keycloak.

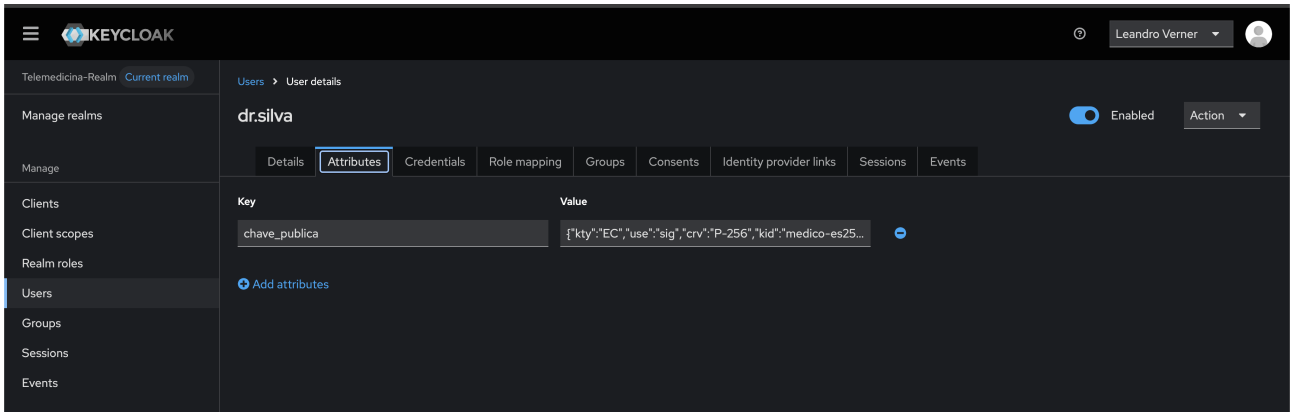


Figura 5 - Atributo chave_publica do usuário dr.silva, gravado pelo backend depois de 'Registrar pública no Keycloak'. O valor é o JWK público ES256 do médico ({"kty":"EC","use":"sig","crv":"P-256","kid":"medico-es256-1",...}) e é a âncora de confiança usada para validar a assinatura do laudo.

3. Inspeção de tokens e estrutura

Os tokens abaixo foram obtidos de um mesmo envio. O JWS foi copiado do portal, antes da cifragem. O JWE foi capturado no Wireshark (interface lo0, filtro tcp.stream eq 20). O instante de emissão do JWS (*iat* = 16:52:08 UTC) coincide com o cabeçalho *Date* da resposta capturada (Sat, 31 Oct 2026 16:52:08 GMT).

3.1 Access token OIDC e chaves

O portal exibe o access token emitido pelo Keycloak (JWS RS256) decodificado, a chave pública estática do servidor (ECDH-ES) e a chave ES256 do médico. O *sub* do access token (e813a525-5294-4e04-8159-4a9e4e9edd90) é o mesmo *sub* do laudo assinado (seção 3.2), e é essa igualdade que o backend verifica. O *azp* é portal-medico-app. O registro da chave do médico retornou *PUT /api/medico/chave-publica → 200*.

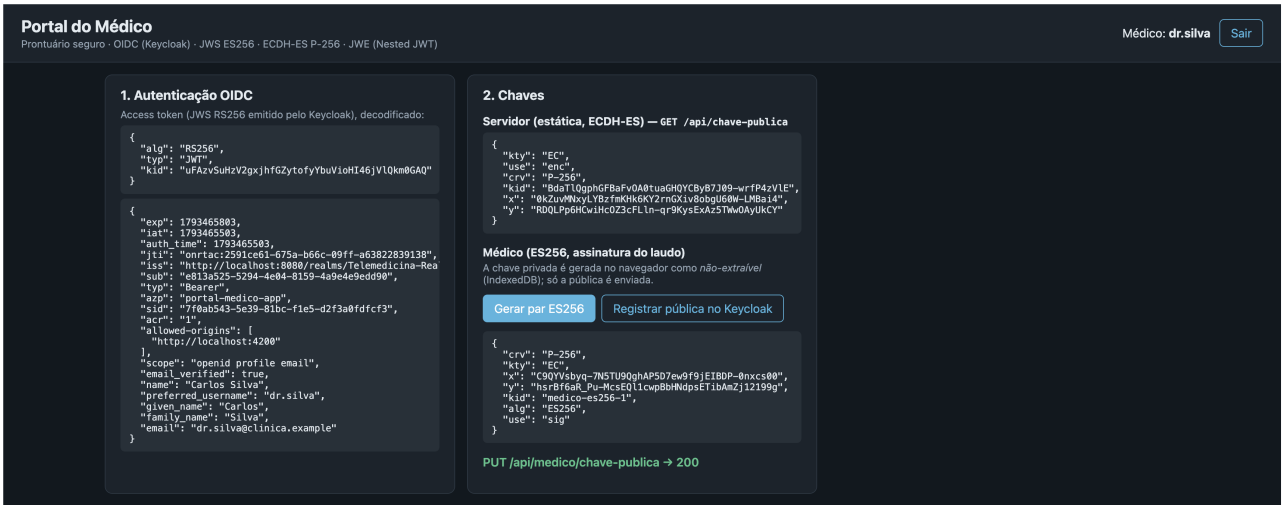


Figura 6 - Portal do Médico, etapas 1 e 2: access token RS256 decodificado (header com kid uFAzvSuHzV2gx...; claims iss, sub, azp, scope), chave pública do servidor obtida em GET /api/chave-publica (EC P-256, use=enc) e chave ES256 do médico (kid medico-es256-1, use=sig).

3.2 JWS assinado (3 partes)

Formato: **Header.Payload.Signature**.


```
eyJhbGciOiJFUzI1NiIsImtpZCI6Im1lZGljby1lczI1Ni0xIiwidHlwIjoiSldUIn0.eyJwYWNpZW50ZV9pZCI6IlAtMTAwMSIsImRpYWdub3N0aWNvIjoiSGlwZXJ0ZW5zw6NvIGFydGVyaWFsIGVzdMOhZ2lvIDEiLCJwcmVzY3JpY2FvIjoiTG9zYXJ0YW5hIDUwbWcsIDF4IGFvIGRpYSIsInN1YiI6ImU4MTNhNTI1LTUyOTQtNGUwNC04MTU5LTRhOWU0ZTllZGQ5MCIsImp0aSI6IjRkNjMyNzcyLWYzYmItNGIxNy1iZTQ2LTdiZmFkMjQyZWNkOCIsImlhdCI6MTc5MzQ2NTUyOCwiZXhwIjoxNzkzNDY1ODI4fQ.pYYdA9FY2KX44BvJ6xkVs6rvMYQzScS3pMHbu5HOLzuSLDD5OPTPsB1wWY40TM3VP1VcRmmhW-uUPUu7YHGBng
```


Parte	Conteúdo decodificado	Tamanho
1. Header	{ "alg": "ES256", "kid": "medico-es256-1", "typ": "JWT" }	67 caracteres
2. Payload	{ "paciente_id": "P-1001", "diagnostico": "Hipertensão arterial estágio 1", "prescricao": "Losartana 50mg, 1x ao dia", "sub": "e813a525-5294-4e04-8159-4a9e4e9edd90", "jti": "4d632772-f3bb-4b17-be46-7bfad242ecd8", "iat": 1793465528, "exp": 1793465828 }	318 caracteres
3. Signature	Assinatura ECDSA P-256 com SHA-256, no formato r||s (32 + 32 bytes).	86 caracteres = 64 bytes

Campo	Significado
alg: ES256	ECDSA sobre P-256 com SHA-256.

Campo	Significado
kid	Identifica a chave do médico que assinou.
sub	ID do médico no Keycloak. O backend exige que seja igual ao *sub* do access token.
jti	Identificador único do laudo, usado contra replay.
iat / exp	Emitido em 2026-10-31 16:52:08 UTC e expira em 16:57:08 UTC (validade de 300 s).

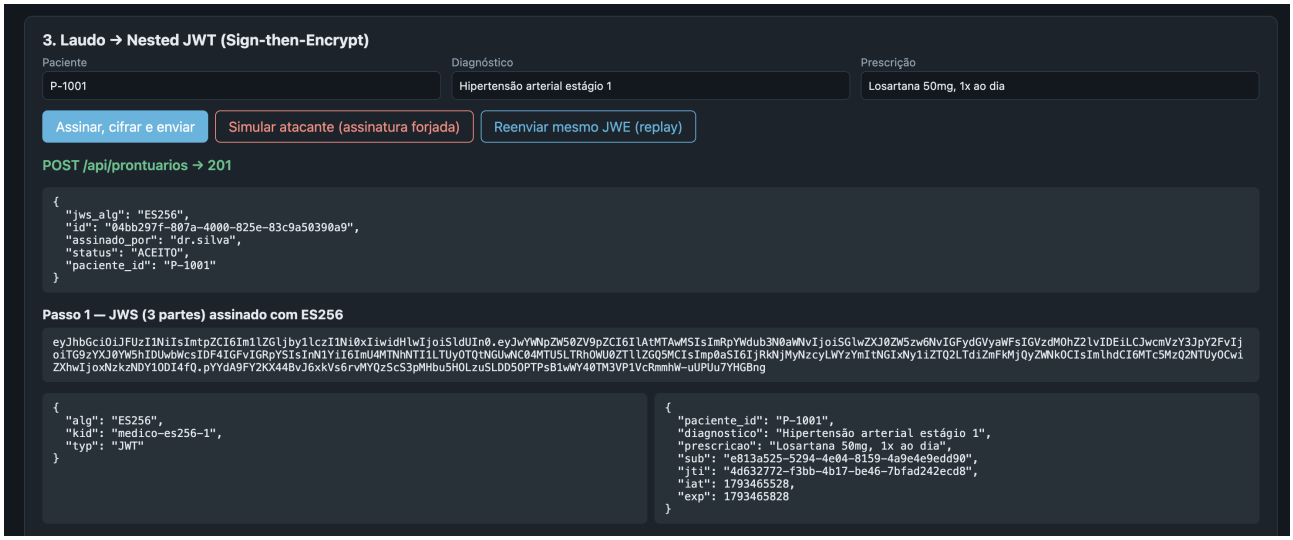


Figura 7 - Portal, passo 1: JWS de 3 partes assinado com ES256, com header e payload decodificados. A resposta do servidor foi POST /api/prontuarios → 201, com status ACEITO, assinado_por dr.silva e jws_alg ES256.

3.3 JWE criptografado (5 partes)

Formato: **ProtectedHeader.EncryptedKey.IV.Ciphertext.AuthTag**. O corpo do POST tem 911 caracteres, valor igual ao *Content-Length: 911* da requisição. Header decodificado (parte 1):

```
{
  "alg": "ECDH-ES",
  "enc": "A128GCM",
  "cty": "JWT",
  "epk": {
    "x": "I5AQ6I0LSpcjtRrRvRHzOMO1TOtY0OAlRSQ_PojAwEc",
    "crv": "P-256",
    "kty": "EC",
    "y": "0TSnInUNevzdbmGJnVEER5MPb1xL7iXELKozTcsVJaI"
  }
}
```

#	Parte	Tamanho (base64url)	Decodificado	Observação
1	Protected Header	238 caracteres	178 bytes	Legível: alg, enc, cty e epk.
2	Encrypted Key	**0**	0	**Vazia.** No ECDH-ES direto a CEK é derivada, não transportada.
3	IV	16 caracteres	12 bytes	96 bits, padrão do AES-GCM.
4	Ciphertext	631 caracteres	473 bytes	O JWS cifrado. Opaco.
5	Authentication Tag	22 caracteres	16 bytes	128 bits, tag do GCM.

Campo do header	Valor	Significado
alg	ECDH-ES	Acordo de chaves Diffie-Hellman sobre curva elíptica em modo direto: a CEK é derivada do segredo compartilhado.

Campo do header	Valor	Significado
enc	A128GCM	Cifra de conteúdo: AES de 128 bits em modo GCM (confidencialidade e integridade).
cty	JWT	O conteúdo cifrado é outro JWT (o JWS). Marca o Nested JWT.
epk	x, y, crv=P-256, kty=EC	Chave pública efêmera do cliente (PA). Com ela o servidor calcula K = nB·PA.

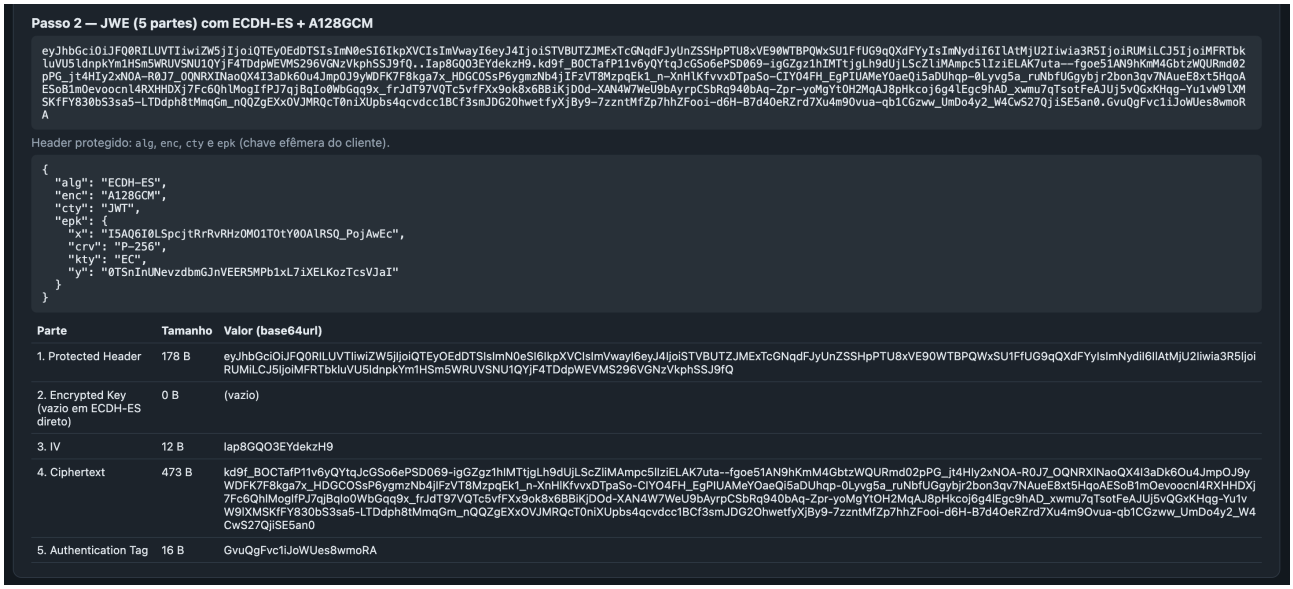


Figura 8 - Portal, passo 2: JWE de 5 partes com ECDH-ES + A128GCM. O header protegido mostra alg, enc, cty e epk. A parte 2 (Encrypted Key) é vazia (0 B). IV e ciphertext coincidem com os bytes capturados no Wireshark (Figura 9): o IV começa com Iap8GQO3... e o ciphertext com kd9f_BOCTaf...

3.4 JWK Set do Keycloak (RFC 7517)

O endereço *http://localhost:8080/realms/Telemedicina-Realm/protocol/openid-connect/certs* publica o conjunto de chaves públicas do realm. Ele contém duas chaves RSA de 2048 bits (expoente público e = AQAB, ou seja, 65537), cada uma com um certificado X.509 em *x5c* cujo subject é Telemedicina-Realm:

kid (início)	kty	alg	use	Função
uFAzvSuHzV2gxjhf...	RSA	RS256	sig	Verifica a assinatura do access token emitido pelo Keycloak.
7Y-lCDr1SEnOSCKy...	RSA	RSA-OAEP	enc	Cifragem de tokens (não usada neste fluxo).

O *kid* do header do access token (Figura 6) é *uFAzvSuHzV2gxjhfGZytofyYbuVioHI46jVlQkm0GAQ*, idêntico ao da chave *sig* acima. O mesmo token aparece no header Authorization da captura (Figura 10). Assim, o Servidor de Prontuários, como resource server, valida o access token com essa chave.

A chave ES256 do médico não está neste JWK Set. O JWKS do realm só lista as chaves do próprio Keycloak. A chave de assinatura do laudo é gerada no navegador e gravada no atributo *chave_publica* do usuário dr.silva, pela Admin API. O backend a lê desse atributo para verificar o JWS. Portanto há dois caminhos de confiança distintos: o JWK Set valida *quem é o chamador* (access token RS256), e o atributo do usuário valida *quem assinou o laudo* (JWS ES256).

4. Análise do tráfego de rede

A captura foi feita no Wireshark, na interface Loopback lo0, com o filtro tcp.stream eq 20. Como o ambiente usa HTTP sem TLS, o Wireshark lê toda a conversa.

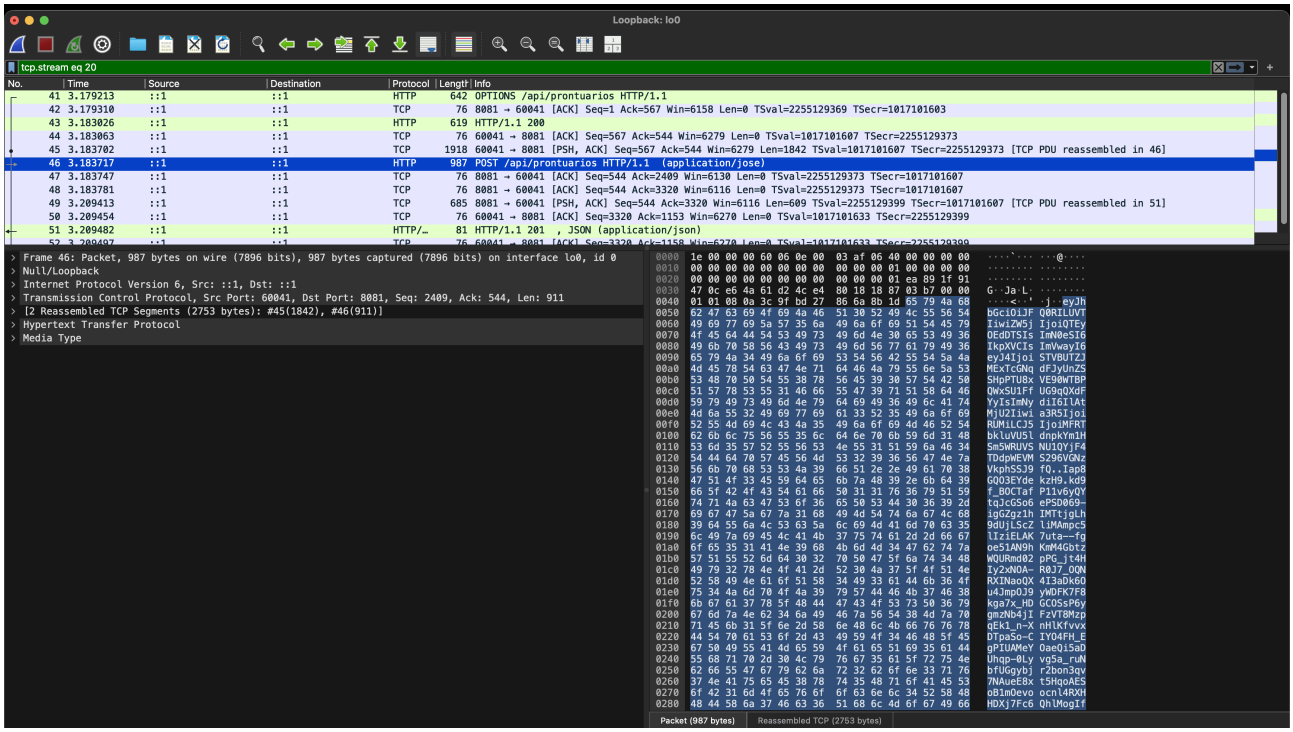


Figura 9 - Wireshark, stream 20. Pacote 41: OPTIONS /api/prontuarios (preflight de CORS) e resposta 200 (pacote 43). Pacote 46: POST /api/prontuarios (application/jose), 911 bytes de corpo. Pacote 51: resposta HTTP 201. No hex, o trecho em azul é o início do JWE; em ASCII ele aparece como eyJhbGciOiJFQ0RILUVTIiwi..., a codificação base64url de {"alg":"ECDH-ES",".

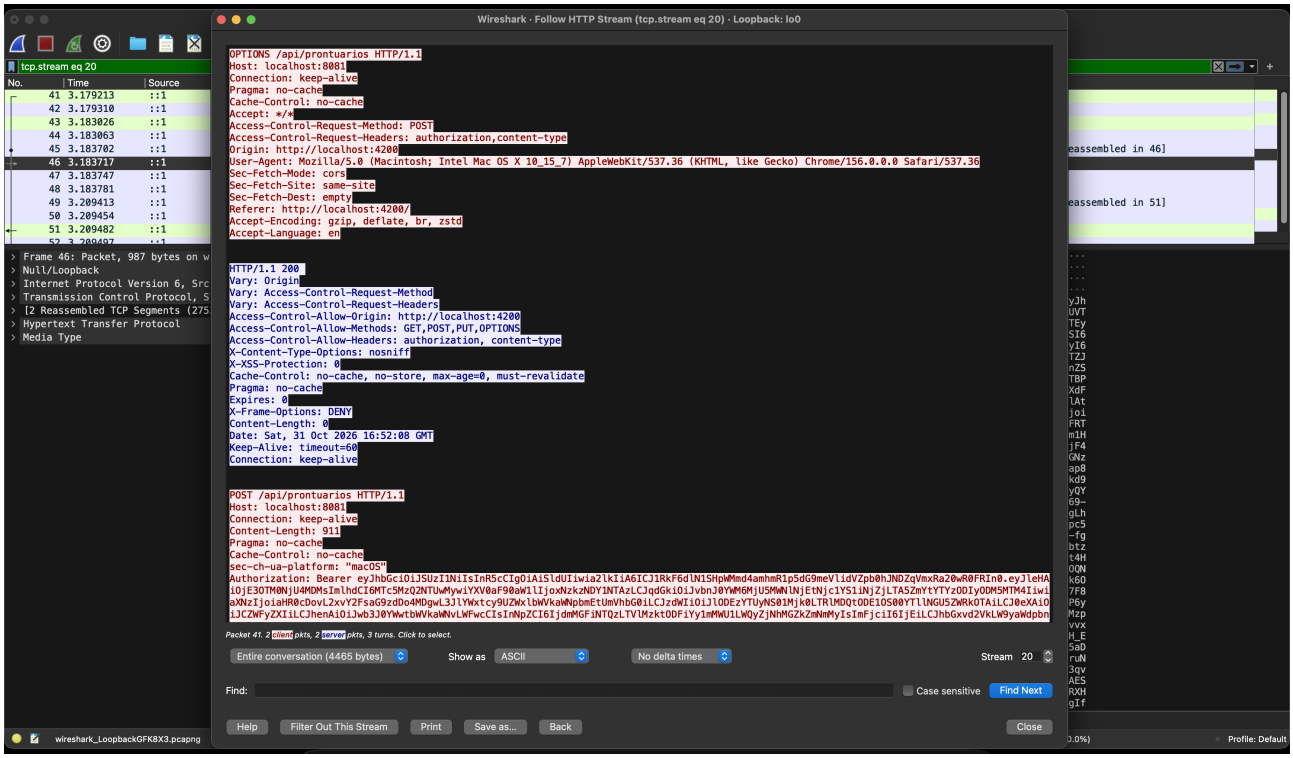


Figura 10 - Follow HTTP Stream: requisição OPTIONS, resposta 200 com os cabeçalhos de CORS (Access-Control-Allow-Origin: http://localhost:4200) e início do POST, com Content-Length: 911 e Authorization: Bearer.

Observações:

- **O corpo do POST é opaco.** Ele contém apenas o JWE. Os dados clínicos (paciente_id, diagnostico, prescricao) estão dentro dos 473 bytes do ciphertext e não aparecem em texto claro na rede.

- **O header do JWE é público por construção.** Ele revela os algoritmos e o epk, que não são segredos. O JWE também não transporta chave: a segunda parte é vazia.

- **Dois tokens trafegam na mesma requisição.** O access token do Keycloak (JWT RS256, no header Authorization) autentica o chamador. O JWE no corpo é o laudo.

- **Limitação do ambiente.** O access token aparece em texto claro na Figura 10, porque o tráfego local é HTTP. Em produção o TLS esconderia esse token. O laudo continua protegido pelo JWE mesmo sem TLS.

5. Análise teórica de segurança

5.1 Como o Diffie-Hellman garante o sigilo do canal

Na curva P-256, a chave pública é o ponto P = n·G, onde n é a chave privada (um escalar) e G é o gerador da curva. O cliente calcula K = nA·PB e o servidor calcula K = nB·PA. Os dois chegam ao mesmo ponto, porque nA·(nB·G) = nB·(nA·G). Um observador da rede vê apenas PA (o epk, no header) e PB (pública). Para obter K ele precisaria de nA ou de nB, o que exige resolver o problema do logaritmo discreto em curvas elípticas (ECDLP), considerado inviável em P-256.

A coordenada x de K passa pelo Concat KDF (RFC 7518, 4.6), que usa SHA-256 e o contexto (enc, apu, apv, tamanho da chave) e produz a CEK de 128 bits do A128GCM. O AES-GCM cifra o JWS e gera a tag de autenticação de 16 bytes, o que dá confidencialidade e integridade ao conteúdo. Como o par (nA, PA) é efêmero e novo a cada envio, cada mensagem usa uma CEK diferente. Isso é visível na captura: dois envios têm epk e ciphertext distintos.

Limite. A chave do servidor (nB) é estática. Se ela vazar, mensagens antigas que um atacante tenha gravado podem ser decifradas. Portanto o sigilo à frente é apenas parcial. A solução seria rotacionar nB ou usar chaves efêmeras também no servidor.

5.2 Por que a assinatura JWS interna é indispensável contra MiTM

O ECDH-ES **não autentica o remetente**. A chave PB é pública, então qualquer pessoa pode gerar um par efêmero, cifrar um conteúdo para o servidor e produzir um JWE válido. O servidor decifra com sucesso, mas isso não prova quem escreveu o laudo. Um atacante no meio do caminho, ou qualquer cliente que consiga chamar a API, poderia forjar ou substituir um laudo.

A assinatura ES256 interna resolve isso:

- **Autenticidade.** O servidor valida o JWS com a chave pública do médico, obtida do Keycloak. Essa chave foi registrada por um usuário autenticado, então o Keycloak é a âncora de confiança. Um laudo assinado com outra chave é rejeitado com 422 (o cenário "Simular atacante" do portal).

- **Integridade.** Alterar o payload invalida a assinatura, e o atacante não possui a privada, que é não extraível e fica no navegador do médico.

- **Não repúdio.** Só o titular da privada ES256 poderia ter produzido a assinatura.

- **Vinculação à sessão e anti-replay.** O *sub* do laudo deve ser igual ao do access token, e *iat/exp* e *jti* impedem o reenvio do mesmo JWE (o "Reenviar mesmo JWE" retorna 422).

Evidência dos ataques simulados. Os dois testes do portal foram rejeitados pelo servidor com HTTP 422. No primeiro, o laudo foi assinado com outra chave ("Simular atacante"): o JWE foi decifrado normalmente, pois o atacante conhece PB, mas a validação do JWS falhou ("assinatura JWS inválida"). No segundo, o mesmo JWE já aceito foi reenviado ("Reenviar mesmo JWE"): o *jti* já havia sido registrado ("jti ausente ou repetido (replay)").

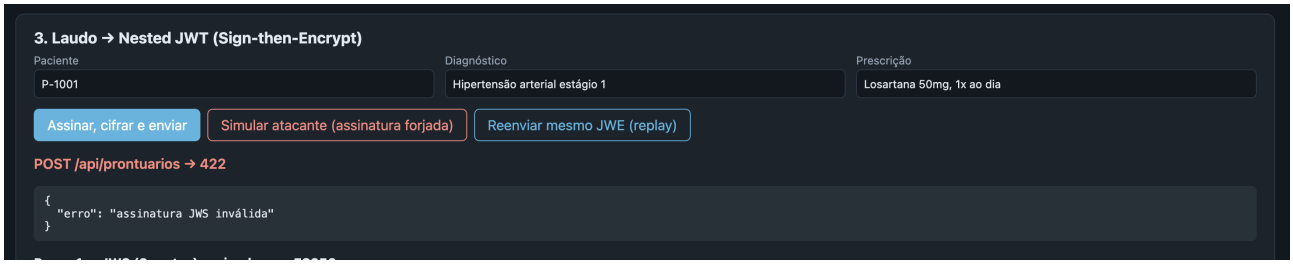


Figura 11 - Simular atacante (assinatura forjada): POST /api/prontuarios → 422, {"erro": "assinatura JWS inválida"}.

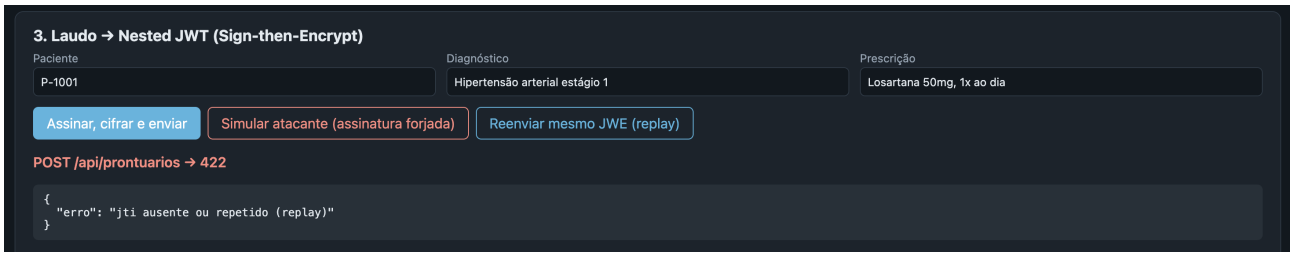


Figura 12 - Reenviar mesmo JWE (replay): POST /api/prontuarios → 422, {"erro": "jti ausente ou repetido (replay)"}.

Por que sign-then-encrypt. Assinar antes de cifrar mantém a assinatura oculta na rede: um observador não vê nem o laudo nem quem o assinou.

5.3 Limitações e melhorias

- **Distribuição de PB.** O cliente obtém PB por HTTP, sem autenticação. Um MiTM ativo poderia trocar PB pela sua própria chave, decifrar o laudo e recifrá-lo para o servidor real. Ele não conseguiria forjar nem alterar o laudo, por causa do JWS, mas conseguiria lê-lo. Em produção, PB deve chegar por um canal autenticado (TLS, ou publicada em um JWK Set assinado ou com pinning).

- **TLS.** O Keycloak em start-dev e o *sslRequired: none* servem apenas ao ambiente local.

- **Armazenamento.** Prontuários e *jti* ficam em memória, e as senhas e o secret são de demonstração.

6. Conclusão

O Nested JWT combinou dois mecanismos complementares. O ECDH-ES, com o JWE, fornece **confidencialidade**: a captura no Wireshark mostra apenas o header público e 473 bytes cifrados. O JWS ES256 fornece **autenticidade, integridade e não-repúdio**, ancorados no Keycloak. Nenhum dos dois substitui o outro. Sem a assinatura, qualquer um que conheça PB poderia enviar laudos. Sem o JWE, os dados clínicos trafegariam em claro.

Referências: RFC 7515 (JWS), RFC 7516 (JWE), RFC 7517 (JWK), RFC 7518 (JWA, 4.6 ECDH-ES e Concat KDF) e RFC 7519 (JWT).